



kubernetes



KubeVirt



목차

쿠버네티스 가상화



목차



목차



과정개요

쿠버네티스 가상화

강사

랩



소개

이 교육은 다음과 같은 목표를 가지고 있음.

1. 쿠버네티스 기반으로 PaaS 가상화 시스템 구축 및 운영
2. 기존에 사용중인 쿠버네티스 플랫폼에 가상화 기능 통합
3. 기존에 사용하던 VMWare, OpenStack, RHV를 쿠버네티스 플랫폼으로 마이그레이션
4. 가상머신 기반 빠른 서비스 배포를 고려중인 서비스 아키텍처 회사



대상

01

쿠버네티스 운영 경험이
있는 사용자

기존 가상머신 환경을
마이그레이션이 필요한
인프라 엔지니어

02

기본적인 리눅스의 표준
가상화 지식이 있는 사용자

Libvirt

Guestfish-tools

oz, dib

03

쿠버네티스를 통한 가상머신
운행을 원하는 서비스 아키
텍처 혹은 인프라 엔지니어



과정개요

현재 CNCF 및 컨테이너 기반 플랫폼의 표준으로 자리 잡은 쿠버네티스(Kubernetes)에 대해 학습하는 과정. 본 교육에서는 쿠버네티스 설치, 구성, 운영을 다루며, OCI 도구와 쿠버네티스의 관계도 함께 살펴본다.

과정은 3일/5일에 따라서 구성이 달라진다. 내용은 다음과 같이 차이가 있다.

길이	설명



강사

강사



강사

이름: 최국현

메일: tang@dustbox.kr, 회신은 다른 메일 주소로 드리고 있습니다. :)

사이트: tang.dustbox.kr

언제든지 질문 및 요청 환영 입니다.



LAB

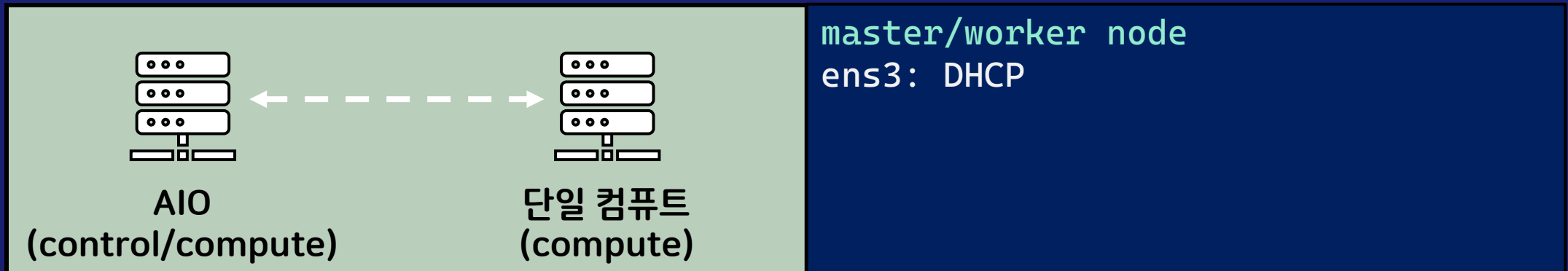
설명



가상머신 구성

기본과정이기 때문에 총 2대의 가상머신을 사용합니다. 컨테이너 및 가상머신 혼합 및 혼용 환경으로 운영 및 CICD는 통합 CICD과정에서 다룰 예정입니다.

네트워크는 ens3만 사용하며, 별도로 추가가 되는 네트워크는 없습니다.



소개

누가 만들었는가?

쿠버네티스 가상화



누가 만들었는가?



Kube-Virt의 탄생 배경

KubeVirt 쿠버네티스 환경에서 가상머신(VM)을 네이티브하게 실행하기 위해 시작된 오픈소스 프로젝트이다.
컨테이너 중심의 Kubernetes 환경에서 기존 VM 기반 워크로드를 함께 운영할 필요성에서 출발하였다.

주요 주도 및 참여 기업

Red Hat

- 프로젝트의 핵심 설계 및 초기 개발을 주도
- Kubernetes 기반 하이브리드 가상화 전략의 일환

IBM, Intel, NVIDIA 등

- 엔터프라이즈 및 하드웨어 벤더들이 커뮤니티 형태로 참여

“KubeVirt는 Red Hat을 중심으로 한 오픈소스 커뮤니티가 Kubernetes를 컨테이너와 VM을 아우르는 플랫폼으로 확장하기 위해 만든 프로젝트이다.”



Kube-Virt의 탄생 배경

현재 많은 가상화 기업들이 쿠버네티스와 통합을 위해서 기존에 사용하던 가상머신 플랫폼을 포기하고 kube-virt 로 통합 혹은 마이그레이션 작업을 많이 이루고 있다.

kube-virt를 사용하는 대표적이 미들웨어 솔루션은 다음과 같다.

1. Redhat OpenShift Virtualization
2. SUSE-VIRT(AKA Harvester)
3. VMWARE Tanzu 현재는 중지, kube-virt 지원하지 않음.



쿠버네티스 가상화 설명

아키텍처

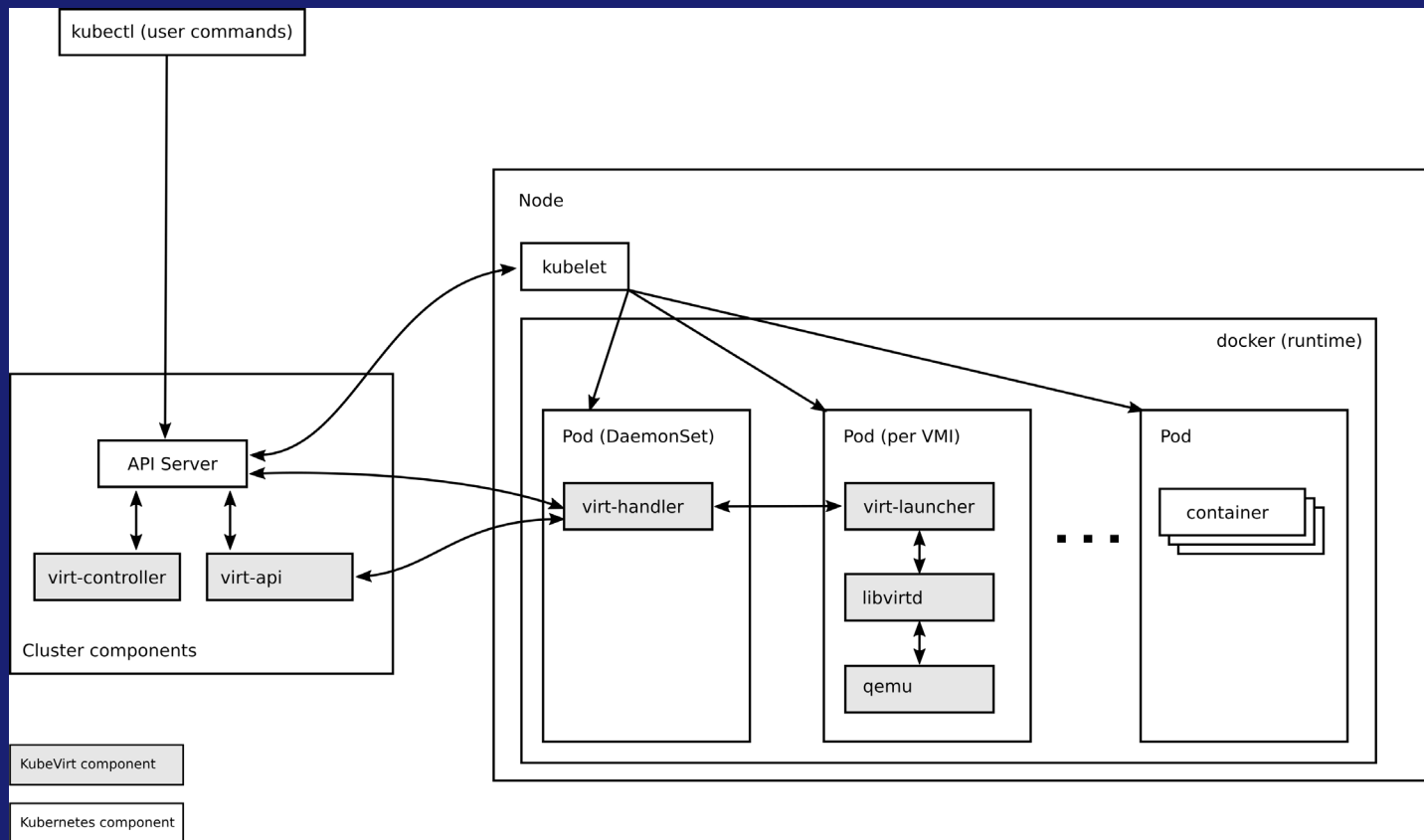
탄생



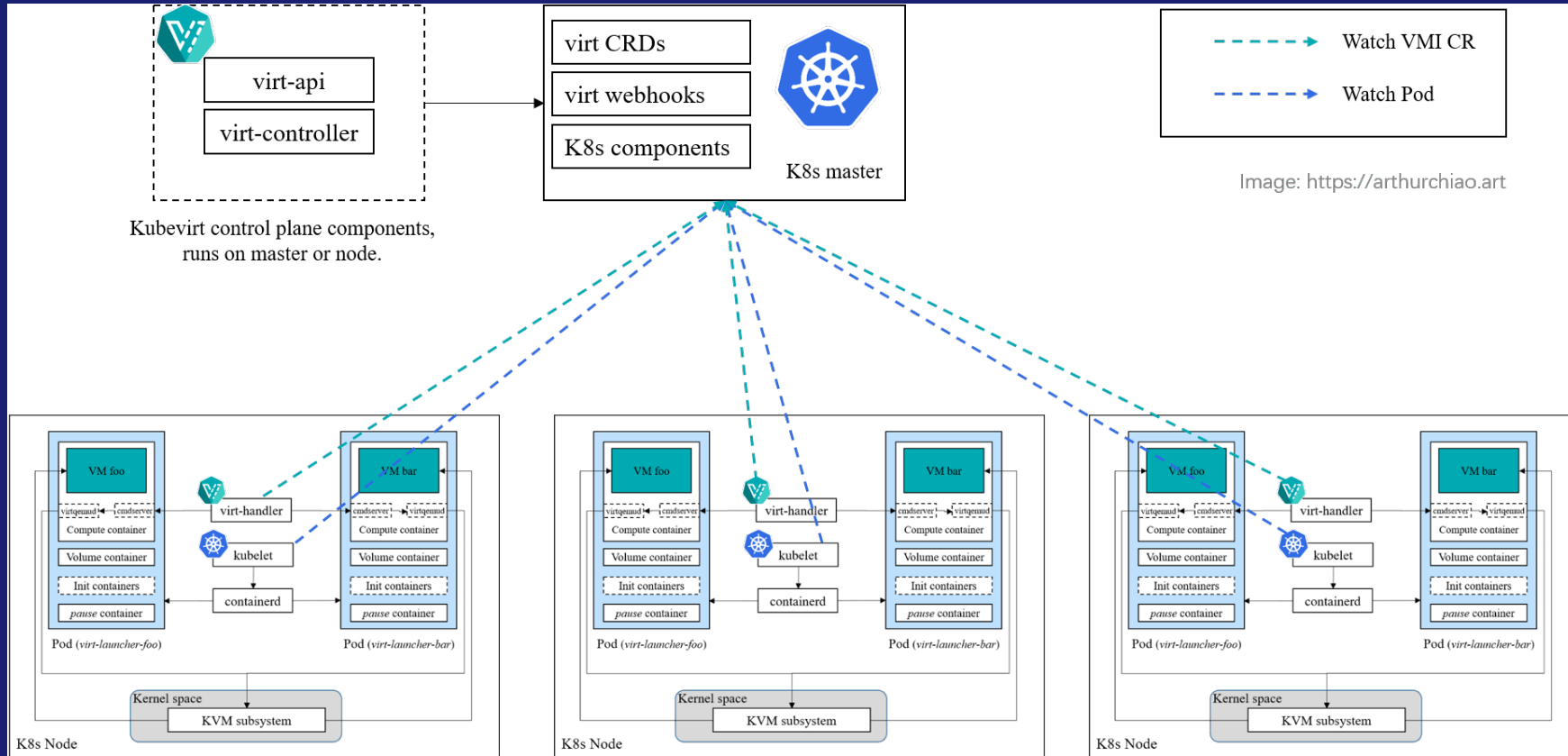
아키텍처



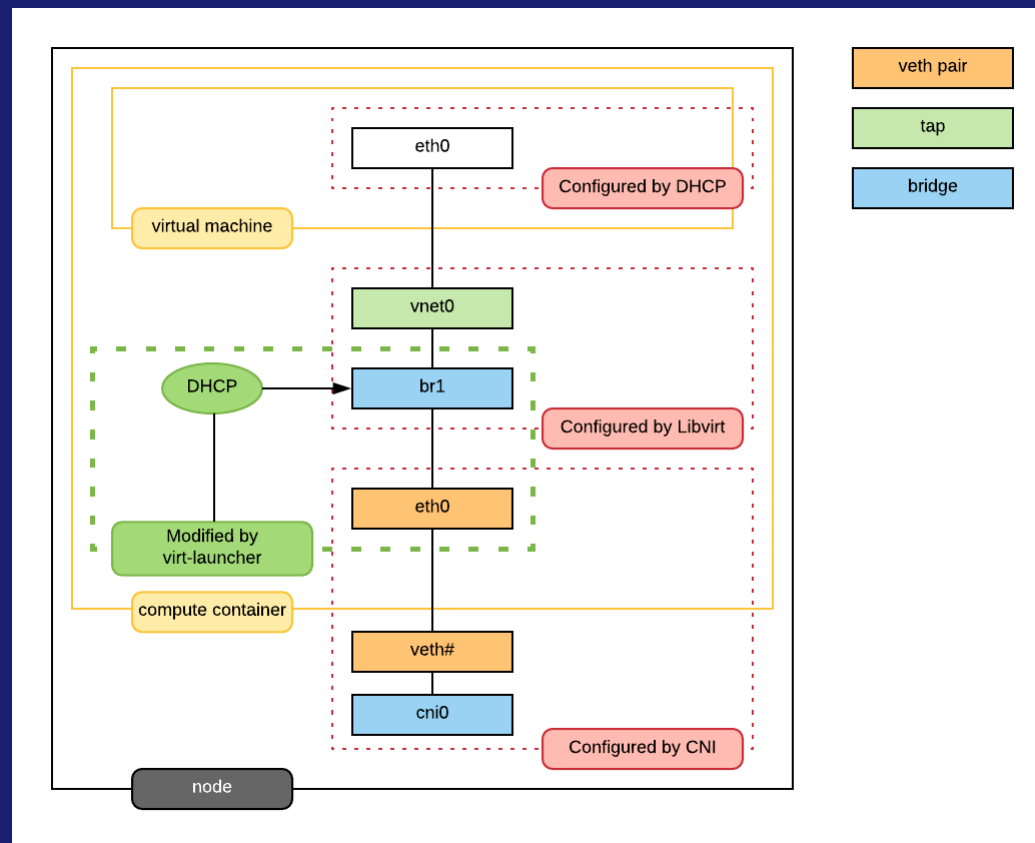
아키텍처 1



아키텍처 2



가상머신 구성 구조



VM 생성 흐름

① 사용자 요청

- 사용자가 VirtualMachine 또는 VirtualMachineInstance(VMI) 생성
- Kubernetes API로 선언적 요청

② 컨트롤 플레인 처리

- virt-controller가 VMI를 감지
- VM 실행을 위한 Pod 스펙 자동 생성



VM 생성 흐름

③ Pod 생성

- Kubernetes가 특수 Pod(virt-launcher Pod) 생성
- 이 Pod 안에:
 - QEMU
 - libvirt
 - VM 디스크(CSI)
 - VM 네트워크(CNI)

④ VM 실행

- Pod 내부에서 QEMU 프로세스로 VM 부팅(이 때, 두 개의 컨테이너가 Pod에서 동작)
- Kubernetes 입장에서는 Pod
- 사용자 입장에서는 VM 컨테이너, libvirt 컨테이너



흐름 아스키 도식

위의 내용을 아스키 도식으로 다음과 같이 구성이 가능하다.

```
[ User/kubectl ]
|
v
[ Kubernetes API Server ]
|
v
[ VirtualMachine/VMI (CRD) ]
|
v
[ virt-controller ]
  (VM → Pod 변환)
|
v
[ virt-launcher Pod ] ← Kubernetes가 인식하는 실행 단위
|
v
[ QEMU+libvirt ]
|
v
[ Virtual Machine(Guest OS) ]
```



설명

쿠버네티스 가상화



쿠버네티스 가상화의 개념

Kubernetes 가상화란

전통적인 하이퍼바이저 중심의 가상머신(VM) 방식이 아니라, 컨테이너(Container) 를 기본 실행 단위로 사용하는
경량 가상화 모델이다.

운영체제 전체를 가상화하지 않고, 커널을 공유하면서 애플리케이션 단위로 격리한다.



기존 가상화와의 차이

VM 가상화

하드웨어 → 하이퍼바이저 → Guest OS → 애플리케이션

쿠버네티스 가상화

하드웨어 → OS 커널 → 컨테이너 → 애플리케이션



핵심 특징

1. 빠른 기동 속도
2. 높은 리소스 밀도
3. 선언적(Declarative) 인프라 운영



전체 구조 개요

Control Plane

- API Server, Scheduler, Controller Manager
- 클러스터 상태와 워크로드를 제어

Worker Node

- 컨테이너 런타임(Containerd, CRI-O 등)
- 실제 애플리케이션(Pod)이 실행되는 영역



가상화 관점에서의 역할

- Pod는 가상 실행 단위
- Namespace는 논리적 격리 공간
- CNI는 가상 네트워크
- CSI는 가상 스토리지

결과적으로 컴퓨트, 네트워크, 스토리지가 모두 소프트웨어 정의(SDI) 형태로 가상화 됨.



왜 VM 가상화가 필요한가?

- 기존 레거시 애플리케이션
- Windows, 커널 의존 서비스
- 컨테이너 전환이 어려운 워크로드



KubeVirt 개요

- 쿠버네티스 위에서 VM을 Pod처럼 실행
- 컨테이너와 VM을 동일한 API로 관리
- Kubernetes 스케줄링, 네트워크, 스토리지를 그대로 활용



하이브리드 가상화의 장점

- 컨테이너+VM 동시 운영
- 점진적 마이그레이션 가능
- 단일 플랫폼으로 운영 복잡도 감소



설치

쿠버네티스 가상화



설치 조건

설치를 위해서 다음과 같은 조건이 필요하다.

1. 최소 한 개의 쿠버네티스 클러스터 혹은 노드
2. 가상화 기능을 지원하는 하드웨어 혹은 가상머신
3. 쿠버네티스 버전 1.30이상 버전 권장
4. 한 개 이상의 스토리지 클래스 권장



설치 시작

이미 쿠버네티스 클러스터가 존재하는 조건으로 다음과 같은 순서로 매니페스트 설치를 진행한다.

```
export KUBEVIRT_VERSION=$(curl -s  
https://api.github.com/repos/kubevirt/kubevirt/releases/latest | grep  
tag_name | cut -d '"' -f 4)  
export KUBEVIRT_VERSION=v1.7.0
```



오퍼레이터 구성

kube-virt는 오퍼레이터 기반으로 설치가 된다. 아래처럼 매니 페스트를 등록한다.

```
# kube-virt operator
```

```
kubectl apply -f
```

```
https://github.com/kubevirt/kubevirt/releases/download/${KUBEVIRT_VERSION}/  
kubevirt-operator.yaml
```

```
# kube-virt CR 구성
```

```
kubectl apply -f
```

```
https://github.com/kubevirt/kubevirt/releases/download/${KUBEVIRT_VERSION}/  
kubevirt-cr.yaml
```



오퍼레이터 설치 확인

올바르게 설치가 완료가 되면 아래 명령어로 확인한다.

```
kubectl -n kubevirt get all  
kubectl get -n kubevirt kubevirt  
kubectl get -n kubevirt pod -w
```



확장 명령어 설치 및 가상화 지원 확인

쿠버네티스에서 가상머신 구성/생성/관리는 아래 명령어로 가능하다. 간단한 조회는 여전히 kubectl 명령어로 가능하다.

```
curl -L -o virtctl
https://github.com/kubevirt/kubevirt/releases/download/${KUBEVIRT_VERSION}/
virtctl-${KUBEVIRT_VERSION}-linux-amd64chmod +x virtctlsudo mv virtctl
/usr/local/bin/
kubectl completion > /etc/profile.d/virtctl.sh
source /etc/profile.d/virtctl.sh
```

```
lsmod | grep kvm
modprobe kvm
modprobe kvm_intel      # Intel
modprobe kvm_amd        # AMD
```



CDI(Containerized Data Importer) 설치

컨테이너 이미지를 관리를 위한 이미지 데이터 가져오기를 설치한다.

```
export CDI_VERSION=$(curl -s  
https://api.github.com/repos/kubevirt/containerized-data-  
importer/releases/latest | grep tag_name | cut -d '"' -f 4)
```

```
kubectl apply -f https://github.com/kubevirt/containerized-data-  
importer/releases/download/${CDI_VERSION}/cdi-operator.yaml
```

```
kubectl apply -f https://github.com/kubevirt/containerized-data-  
importer/releases/download/${CDI_VERSION}/cdi-cr.yaml
```



CDI(Containerized Data Importer) 확인

올바르게 설치가 되었는지 확인 및 이미지 업로드 한다.

```
kubectl -n cdi get pods  
virtctl image-upload --help
```

```
mkdir -p ~/kubevirt-imagescd ~/kubevirt-images  
curl -LO https://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-  
disk.img  
ls -lh cirros-0.6.2-x86_64-disk.img
```



CDI(Containerized Data Importer) 확인

올바르게 설치가 되었는지 확인 및 이미지 업로드 한다.

```
virtctl image-upload dv cirros-dv --size=2Gi \  
--image-path=cirros-0.6.2-x86_64-disk.img --namespace=cdi \  
--storage-class=standard \  
--uploadproxy-url=https://cdi-uploadproxy.default.svc.cluster.local:443 \  
--insecure
```

```
kubectl get sc  
kubectl get dv  
kubectl get pvc
```



CDI(Containerized Data Importer) 확인

올바르게 DNS질의가 되지 않는 경우 ClusterIP로 접근한다.

```
kubectl get -n cdi svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
cdi-uploadproxy	ClusterIP	172.20.84.96	<none>	443/TCP

```
virtctl image-upload dv cirros-dv --size=2Gi \  
--image-path=cirros-0.6.2-x86_64-disk.img --namespace=cdi \  
--storage-class=standard \  
--uploadproxy-url=https://172.20.84.96:443 \  
--insecure
```



스토리지 프로파일 설명(sp)

스토리지 프로파일은 VM 생성 시, 사용하는 디스크 자원이다. 이 자원을 통해서 가상머신이 사용할 디스크를 생성한다. 스토리지 프로파일은 StorageClass, PersistentVolumeClaim이 올바르게 구성이 되어 있어야 사용이 가능하다.



스토리지 프로파일 생성(sp)

테스트용도 가상머신이 생성 될 스토리지 프로파일을 생성해야 한다. 없는 경우 올바르게 가상머신이 생성이 되지 않는다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: StorageProfile
metadata:
  name: nfs-client
spec:
  claimPropertySets:
  - accessModes:
    - ReadWriteMany
    volumeMode: Filesystem
```

EOF



스토리지 클래스 변경(sc)

가상머신이 NFS 기반으로 생성이 되는 경우, RWX로 구성을 권장한다. 아래와 같이 변경을 권장한다.

```
kubectl patch storageprofile nfs-client --type='merge' -p \
'{"spec":{"claimPropertySets":[{"accessModes":["ReadWriteOnce"],"volumeMode":"Filesystem"}]}}'
```



가상머신 생성 1

간단하게 가상머신을 생성 후 올바르게 구성이 되었는지 확인한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: cirros-vm
spec:
  running: true
  template:
    spec:
      domain:
        cpu:
          cores: 1
```



가상머신 생성 2

위의 내용 계속 이어서...

```
resources:
  requests:
    memory: 256Mi
  devices:
    disks:
      - name: rootdisk
        disk:
          bus: virtio
  volumes:
    - name: rootdisk
      persistentVolumeClaim:
        claimName: cirros-dv
```



가상머신 생성 확인

올바르게 생성이 되었는지 확인한다.

```
kubectl get vmi
```

```
cirros-vm    Running
```

```
kubectl get pvc
```

```
cirros-dv          Bound      pvc-9f2ac056-2624-4e2a-92b6-8fff8a34234d
2276332667    RWX          nfs-client      <unset>          3h35m
```

```
virtctl console cirros-vm
```

```
login: cirros
```

```
password: gocubsgo
```



기본 명령어 학습

기본 자원 다루기



시작 전...

쿠버네티스 가상화는 두 가지 명령어를 사용한다.

1. 쿠버네티스 기본 명령어 kubectl
2. 쿠버네티스 가상화 명령어 virtctl

이 과정에서는 이 두 개 명령어가 지원하는 기능에 대해서 확인한다. 명령어 활용 부분은 뒤에 가상머신/스토리지/네트워크 부분에서 다루기 다시 다룬다.



명령어 구별

쿠버네티스 가상화를 사용하면 두 가지 명령어를 통해서 관리가 필요하다.

1. kubectl
2. virtctl

kubectl는 본래 용도처럼 쿠버네티스에서 구성 및 관리된 자원을 관리하는 명령어이다. 일반적으로 사용하는 PV/PVC/SC 그리고 VM/VMI/DC와 같은 자원으로 분리가 된다.

kubectl를 통해서 아래와 API 자원을 분리하여 확인이 가능하다.



API 자원 확인

API에서 다음과 같이 자원이 분류 및 확인이 가능하다.

```
kubectl api-resources | grep kubevirt.io | awk 'BEGIN { printf "%-35s %-10s\n", "FULL", "SHORT" } { printf "%-35s %-10s\n", $1, $2 }'
```

FULL	SHORT
cdiconfigs	cdi.kubevirt.io/v1beta1
cdis	cdi,cdis
dataimportcrons	dic,dics
datasources	das
datavolumes	dv,dvs
objecttransfers	ot,ots



가상머신 확인 1

제일 많이 사용하는 명령어는 바로 가상머신이 올바르게 동작하는가? 이다. 가상머신 동작상태를 확인하기 위해서 아래와 같이 명령어로 확인이 가능하다. 다만, 두 가지 자원에 대해서 구별해야 한다.

1. `VirtualMachine (vm)`

2. `VirtualMachineInstance (vmi)`

두 개는 비슷하게 보이지만, 다른 형식으로 자원을 관리 및 구별하고 있다.

`VirtualMachine`은 가상머신이 미사용중인 상태이어도 화면에 출력이 된다. 즉, 인스턴스 목록과 비슷하다. 이 안에는 가상머신이 실행 시 사용하는 일종의 가상머신 사양을 가지고 있다.

```
kind: VirtualMachine
spec:
  running: true
```



가상머신 확인 2

VirtualMachineInstance (vmi)는 실제로 동작중인 가상머신 자원을 말한다. 이러한 이유로 뒤에 Instance라고 단어가 붙어있다.

역할

- 실제 QEMU/KVM 프로세스
- 노드에 스케줄링됨
- CPU/메모리/네트워크를 실제로 소비

```
kind: VirtualMachineInstance
status:
  phase: Running
```

특징

- VM이 켜지면 자동 생성
- VM을 끄면 자동 삭제
- Pod와 매우 유사한 라이프사이클



가상머신 확인 3

위의 내용을 정리하면 다음과 같다.

VirtualMachine (vm)

└ spec.running: true



VirtualMachineInstance (vmi) 생성



노드에 스케줄링



QEMU/KVM(Libvirt)



가상머신 확인 4

아래와 같이 기능 확인이 가능하다. 강제로 patch를 통해서 true → false으로 변경한다.

```
kubectl get vm
```

NAME	AGE	STATUS	READY
cirros-vm	3h54m	Running	True

```
kubectl get vmi
```

NAME	AGE	PHASE	IP	NODENAME	READY
cirros-vm	3h54m	Running	172.16.236.151	kubevirt.example.com	True

```
kubectl patch vm cirros-vm --type merge -p '{"spec":{"running":false}}'
```

```
kubectl get vm
```

NAME	AGE	STATUS	READY
cirros-vm	3h55m	Stopped	False

```
kubectl get vmi
```

NAME	AGE	PHASE	IP	NODENAME	READY
cirros-vm	3h55m	Succeeded		kubevirt.example.com	False



가상머신 확인 마지막

이 기능을 오픈스택 및 쿠버네티스와 비교하면 다음과 같다.

개념	KubeVirt	Kubernetes	OpenStack
설계/의도	VM	Deployment	Server
실행 인스턴스	VMI	Pod	Instance
전원 제어	VM	Replica	nova start/stop



DV 1

DV, DataVolume은 가상머신 생성 시, 사용할 볼륨/디스크 생성한다. 오픈스택과 비교하면 Glance에서 Cinder로 이미지 전환하는 과정이다.

```
kubectl get dv
```

NAME	PHASE	PROGRESS	RESTARTS	AGE
cirros-dv	Succeeded	100.0%	0	6m



DV 2

DV, DataVolume

가상머신 생성 시, 사용할 볼륨/디스크 생성한다. 오픈스택과 비교하면 Glance에서 Cinder로 이미지 전환하는 과정이다.

```
kubectl get dv
```

NAME	PHASE	PROGRESS	RESTARTS	AGE
cirros-dv	Succeeded	100.0%	0	6m



PVC

PVC

DV에서 구성된 데이터를 쿠버네티스로 가져오면, 해당 정보는 Pod에서 사용이 가능하도록 PVC정보로 변경이 된다.

OpenStack 비교

- PVC=Cinder Volume (또는 Nova에서 붙이는 Block Device)
- StorageClass=Cinder backend 타입(NFS/Ceph/LVM 같은 스토리지 정책)

```
kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY
cirros-dv	Bound	pvc-7f1b2d86-5d1b-4bde-a0f0-2d2c7a3c9f1a	2Gi



DATAVOLUME(dv)

DataVolume

올바르게 구성이 되면, 데이터 볼륨을 통해서 인스턴스가 올바르게 구성 및 사용 중인지 확인이 가능하다.

```
kubectl describe dv cirros-dv
```

```
Name:          cirros-dv
```

```
Namespace:     default
```

```
Phase:         Succeeded
```

```
Progress:      100.0%
```

```
Events:
```

```
Normal UploadScheduled ... Upload pod scheduled
```

```
Normal ImportSucceeded ... DataVolume completed successfully
```



VMI

VMI(VirtualMachineInstance)

OpenStack 비교

- VMI=실제 하이퍼바이저 위에서 돌고 있는 인스턴스 런타임
- Nova의 instance state가 Running일 때 대시보드 혹은 CLI 출력

```
kubectl get vmi
```

NAME	AGE	PHASE	IP	NODENAME
cirros-vm	4m	Running	10.244.0.23	kube-virt



VM 1

VM(VirtualMachine)

OpenStack 비교

- openstack server stop/start와 거의 동일한 체감
- 차이: KubeVirt는 VMI 인스턴스 런타임이라 중지 시 VMI가 없어짐

```
kubectl patch vm cirros-vm --type merge -p '{"spec":{"running":false}}'
```

```
kubectl get vm,vmi
```

NAME	AGE	STATUS
virtualmachine/cirros-vm	5m	Stopped

```
No resources found in default namespace.
```



VM 2

VM(VirtualMachine)

아래 명령어는 인스턴스를 patch를 통해서 실행 상태로 변경.

```
kubectl patch vm cirros-vm --type merge -p '{"spec":{"running":true}}'  
kubectl get vmi
```

NAME	AGE	PHASE	IP	NODENAME
cirros-vm	10s	Running	10.244.0.24	kube-virt



SNAPSHOT 설명

VirtualMachineSnapshot

다른 PaaS와 동일하게 Snapshot 생성 및 관리가 가능하다. 여기서는 생성부분은 다루지 않고 간단하게 확인 및 비교 한다.

OpenStack 비교

- openstack volume snapshot create 또는 openstack server image create와 비슷
- 구현은 스토리지/CSI 기능에 의존(백엔드가 지원해야 스냅샷 가능)
- 생성 및 복구는 YAML 통해서 가능
- 조회는 kubectl 명령어를 통해서 가능



SNAPSHOT 기능 활성화

기본값으로 설치한 경우, Snapshot이 동작하지 않는다. 사용하기 위해서는 Snapshot 기능을 활성화 해야 한다. 활성화를 위해서는 아래와 같이 CR에 다음과 같이 기능을 수정한다.

일반적인 운영에서는 다음과 같은 기능을 활성화 한다.

```
kubectl -n kubevirt patch kubevirt kubevirt --type='merge' -p '
{
  "spec": {
    "configuration": {
      "developerConfiguration": {
        "featureGates": ["LiveMigration", "CPUManager", "Snapshot"]
      }
    }
  }
}'
```



SNAPSHOT 생성

명령어로는 스냅샷 생성은 어렵다. 생성을 위해서는 아래와 같이 YAML으로 작성 및 구성해야 한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineSnapshot
metadata:
  name: cirros-snap
  namespace: default
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: cirros-vm
```

EOF



SNAPSHOT 목록 확인

스냅샷이 구성이 되면 아래와 같이 확인이 가능하다.

```
kubectl get virtualmachinesnapshot
```

NAME	SOURCEKIND	SOURCENAME	PHASE	READYTOUSE	
CREATIONTIME	ERROR				
cirros-snap	VirtualMachine	cirros-vm	Succeeded	true	3m59s



SNAPSHOT 복구

명령어로는 스냅샷 생성은 어렵다. 생성을 위해서는 아래와 같이 YAML으로 작성 및 구성해야 한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: snapshot.kubevirt.io/v1beta1
kind: VirtualMachineSnapshot
metadata:
  name: cirros-snap
  namespace: default
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: cirros-vm
```

EOF



SNAPSHOT 오픈스택과 비교

오픈스택과 기능을 비교하면 아래와 같다.

OpenStack	KubeVirt
<code>openstack server image create</code>	VirtualMachineSnapshot
<code>openstack volume snapshot create</code>	VolumeSnapshot
<code>snapshot → server restore</code>	VirtualMachineRestore
명령형 CLI	선언형 CRD



CLONE/POOL 설명

여러 개의 인스턴스를 생성 및 관리 시 사용한다. 오픈스택과 다른 부분이 많기 때문에, 사용 시 몇몇 부분은 주의 해야 한다.

1. 공유 PVC로 간단히(랩 용도)
2. DataSource+개별 VM 생성(또는 자동화 스크립트/오퍼레이터, 운영 용도)
3. Clone 여러 개+라벨 그룹 운영(운영 용도)

OpenStack 비교

- 템플릿 기반 복제=Glance 이미지로 새 서버 만들기과 유사
- KubeVirt는 VM 정의+디스크 복제가 한 덩어리로 움직임

자세한 내용은 가상머신 관리 부분에서 다시 다루도록 한다. 여기서는 간단한 예제만 다룬다.



테스트를 위한 POOL 생성

```
cat <<'EOF' | kubectl apply -f -
apiVersion: clone.kubevirt.io/v1beta1
kind: VirtualMachineClone
metadata:
  name: cirros-clone
  namespace: default
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: cirros-vm
  target:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: cirros-vm-copy
EOF
```



테스트 POOL 확인 및 검증

생성이 완료가 되면, 아래 명령어로 간단하게 확인이 가능하다. 하지만, 현재 이미지는 쿠버네티스 가상화의 기능을 완전히 제공하지 않는다.

```
kubectl get vmclone
```

NAME	PHASE	SOURCEVIRTUALMACHINE	TARGETVIRTUALMACHINE
cirros-clone		cirros-vm	cirros-vm-copy

```
kubectl describe vmclone cirros-clone
```

Events:

Type	Reason	Age	From
Normal	VMVolumeSnapshotsInvalid	6m53s (x2 over 6m53s)	clone-controller

Virtual Machine volume rootdisk does not support snapshots



현재 사용중인 이미지는 rootdisk 공유가 안되는 일반 가상 머신 이미지. 이미지 빌드에서 다시 설명

골든 이미지 등록(dv)

```
cat <<'EOF' | kubectl apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataVolume
metadata:
  name: cirros-http-dv
  namespace: default
spec:
  pvc:
    storageClassName: nfs-client
    accessModes: ["ReadWriteMany"]
    volumeMode: Filesystem
  resources:
    requests:
      storage: 2Gi
```

자세한 설명은 운영에서 다룹니다!

계
속



골든 이미지 등록(dv)

```
source:  
  http:  
    url: https://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-  
disk.img  
EOF
```



골든 기반 인스턴스 생성 1(vm)

```
cat <<'EOF' | kubectl apply -f -
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: cirros-01
  namespace: default
spec:
  running: true
  template:
    spec:
      domain:
        resources:
          requests:
            memory: 256Mi
```

간단하게 강사 설명 :)

계
속



골든 기반 인스턴스 생성 2(vm)

```
    devices:
      disks:
        - name: rootdisk
          disk:
            bus: virtio
    volumes:
    - name: rootdisk
      persistentVolumeClaim:
        claimName: cirros-http-dv
```

EOF

```
kubectl get vm -w
```

NAME	AGE	STATUS	READY
cirros-01	1s	Starting	False
cirros-vm	6h28m	Running	True



가상머신 풀 설명(vmpool)

가상머신 웅덩이(Pool)에서는 여러 개의 가상머신을 스케일링이 가능하도록 한다. clone은 기본적으로 복제가 주요 목적이기 때문에 스케일링(scaling)이 불가능하다.

쿠버네티스의 장점인 스케일링 기능을 사용하기 위해서 VMP(VirtualMachinePool)를 통해서 이 기능을 구현해야 한다. 아래와 같이 구성 및 생성이 가능하다.

여기서는 간단하게 테스트만 한다.



가상머신 풀 간단한 예제 1(vmpool)

```
cat <<'EOF' | kubectl apply -f -
apiVersion: pool.kubevirt.io/v1alpha1
kind: VirtualMachinePool
metadata:
  name: cirros-pool
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      kubevirt.io/vmpool: cirros-pool
```



가상머신 풀 간단한 예제 2(vmpool)

```
virtualMachineTemplate:  
  metadata:  
    labels:  
      kubevirt.io/vmpool: cirros-pool  
      app: cirros  
  spec:  
    running: true  
    template:  
      spec:  
        domain:  
          cpu:  
            cores: 1
```



가상머신 풀 간단한 예제 3(vmpool)

```
resources:
  requests:
    memory: 256Mi
devices:
  disks:
    - name: rootdisk
      disk:
        bus: virtio
volumes:
  - name: rootdisk
    persistentVolumeClaim:
      claimName: cirros-dv
```

EOF



가상머신 풀 확인 3(vmp)

아래와 같이 확인 및 스케일링이 가능하다.

```
kubectl get vmpool
```

NAME	DESIRED	CURRENT	READY	AGE
cirros-pool	2	2		48s

```
kubectl scale vmpool cirros-pool --replicas=3
```

```
virtualmachinepool.pool.kubevirt.io/cirros-pool scaled
```

```
kubectl scale vmirs cirros-pool --replicas=2
```

```
error: no objects passed to scale
```

```
virtualmachineinstancereplicaset.kubevirt.io "cirros-pool" not found
```

vmirs부분은 운영에
서 따로 다룬다.



CLONE/POOL 비교

방식	Pool 사용	디스크 분리	실습용	운영	OpenStack 대응
Pool + 공유 PVC	가능	불가능	가능	비권장	동일 이미지 NFS 부팅
Pool + 개별 PVC	비권장	불가능(자동)	비권장	비권장	직접 지원 없음
Clone 여러 개 + 라벨	불가능	가능	가능	권장	이미지로 서버 다수 생성
DataSource + VM 템플릿	불가능	가능	비권장	권장	Glance Golden Image
Harvester	가능	가능	권장	권장	Horizon 기반 운영



데이터소스(DataSource,ds) 설명

DataSource=KubeVirt식 Glance 이미지 포인터.

- 실제 데이터를 들고 있지 않음
- "이 PVC가 골든 이미지"라고 참조만 제공.
- 여러 VM이 이걸 기준으로 각자 디스크를 생성

이를 통해서 가상머신 생성이 가능하다. 또한 제공하는 소스 형식이 다양하게 제공이 된다.

1. HTTP
2. PVC
3. Docker Registry

자세한 부분은 가상머신 운영에서 다루도록 한다.



데이터소스 생성(ds)

아래와 같이 데이터소스(DataSource)에 접근이 가능하다. 먼저, 테스트 용도로 간단하게 생성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataSource
metadata:
  name: cirros-golden
  namespace: default
spec:
  source:
    pvc:
      name: cirros-dv
      namespace: default
```

EOF



데이터소스 조회(ds)

생성 후, 아래와 같이 조회가 가능하다.

```
kubectl get datasources
```

NAME	AGE
cirros-golden	7s



이미지 자동 갱신(DATA IMPORT CRON, dic)

이미지를 주기적으로 업데이트가 필요한 경우가 있다. 이러한 경우 DIC를 활용하여 주기적으로 Goden 이미지를 업데이트가 가능하다.

다만, DIC를 활용하기 위해서는 최소 한 개의 컨테이너 레지스트리 서버 혹은 외부에서 다운로드가 가능한 웹 서버가 필요하다.



이미지 자동 갱신 확인(dic)

사용중인 이미지를 자동으로 갱신이 되도록 하기 위해서 CRON를 구성하여 자동으로 업데이트 되게 한다.

```
kubectl get dataimportcron  
kubectl describe dic ubuntu-cron
```



이미지 자동 갱신 예제 1(dic)

```
cat <<'EOF' | kubectl apply -f -
apiVersion: cdi.kubevirt.io/v1beta1
kind: DataImportCron
metadata:
  name: cirros-cron
  namespace: default
spec:
  schedule: "*/30 * * * *"
  managedDataSource: cirros-latest
  template:
    spec:
```



이미지 자동 갱신 예제 2(dic)

```
source:
  registry:
    url: docker://quay.io/kubevirt/cirros-container-disk-
demo:20260201_54046b51c9-amd64
  storage:
    storageClassName: nfs-client
    accessModes:
      - ReadWriteMany
    volumeMode: Filesystem
  resources:
    requests:
      storage: 2Gi
```

EOF



이미지 자동 갱신 확인(dic)

아래 명령어로 확인이 가능하다.

```
kubectl get dataimportcron
```

```
NAME          FORMAT
```

```
cirros-cron
```

```
kubectl describe dic cirros-cron
```

```
Spec:
```

```
  Managed Data Source:  cirros-latest
```

```
  Schedule:             */30 * * * *
```

```
  Template:
```

```
    Source:
```

```
      Registry:
```

```
        URL:  docker:// quay.io/kubevirt/cirros-container-disk-  
demo:20260201_54046b51c9-amd64
```



스토리지 프로파일

가상머신이 생성 시 사용할 PVC를 선언한다.

```
kubectl get storageprofile
```

NAME	AGE
nfs-client	6h57m



이미지 업로드 토큰 설명(utr)

간단하게 모든 이미지 업로드를 위한 토큰.

UploadTokenRequest는 사용자가 디스크 이미지를 안전하게 업로드할 수 있도록 일회성 업로드 토큰(URL)을 발급하는 CDI 보안 메커니즘이다.

토큰은 사용자가 생성하는 자원은 아니며, `virtctl image-upload`와 같은 명령어를 사용 시 자동으로 생성된

kind: UploadTokenRequest



이미지 업로드 토큰 설명(utr)

왜 필요한가?

KubeVirt/CDI에서 이미지 업로드는...

1. 내부 서비스(cdi-uploadproxy)를 통해서만 가능
2. 아무나 접근하면 안 됨
3. 업로드 권한은 짧은 시간+1회성이 이상적

항목	설명
목적	이미지 업로드용 임시 인증 토큰 발급
대상	DataVolume (Upload 타입)
유효성	짧은 TTL + 1회성
보안	ServiceAccount/RBAC 기반
방법	virtctl image-upload, UI, 자동화 도구



이미지 업로드 토큰 동작 순서 설명

① UploadTokenRequest 생성

- 사용자가 “이 DV에 업로드하고 싶다” 요청
- CDI가 업로드 전용 토큰 생성

② 토큰 반환

- API 서버가 Upload URL + Token 제공
- 내부 서비스(cdi-uploadproxy) 주소 포함

③ 이미지 업로드

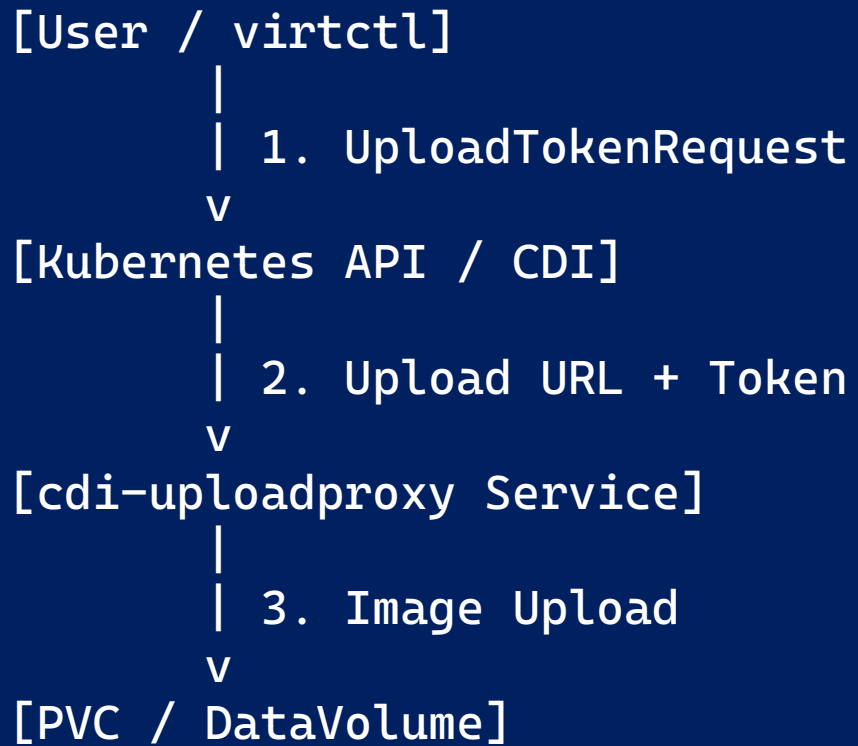
- 클라이언트가 토큰으로 인증
- qcow2 / raw 이미지 업로드

④ 토큰 만료

- 업로드 완료 or 시간 초과 시 자동 폐기



이미지 업로드 토큰 동작 순서



가상머신 마이그레이션(vmim) 설명

VMIM은 실행 중인 VM(VMI)을 중단 없이 다른 노드로 이동시키는 라이브 마이그레이션 리소스다. 이 기능은 다른 솔루션도 사용하는 마이그레이션과 동일하다.

VMIM은 다음과 같은 기능을 제공한다.

1. 실행 중인 VirtualMachineInstance(VMI) 다른 노드로 이동
2. 이동 시 다운타임 최소화
3. Kubernetes 컴퓨트 노드로 이동(클러스터 이동은 미지원)



가상머신 마이그레이션(vmim) 활용

마이그레이션 기능 활용 방법은 두 가지가 있다.

1. 명령어로 실행
2. YAML으로 선언 및 실행

```
kubectl get vmim
```

NAME	PHASE	VMI
cirros-migration	Succeeded	cirros-vm

Multus나 혹은 Bridge네트워크로 변경해야 한다.

```
virtctl migrate cirros-vm
```

Message: cannot migrate VMI which does not use masquerade, bridge with kubevirt.io/allow-pod-bridge-network-live-migration VM annotation or a migratable plugin to connect to the pod network



가상머신 마이그레이션 네트워크

쿠버네티스 가상화에서는 네트워크를 다음과 같이 구성을 권장한다.

네트워크 방식	브릿지 직접 생성	Migration
Pod + bridge	아니요	기본적으로 불가능
Pod + masquerade	아니요	가능
Multus + Linux bridge	가능	구성에 따라 불가능
Multus + SR-IOV	PF/VF 필요	미지원



가상머신 복제(vmirs) 설명 1

VMIRS는 동일한 스펙의 VMI를 지정한 개수만큼 유지하는 VMI 전용 ReplicaSet이다. 그러면, VMIRS가 어떠한 기능을 구현하고 기존 쿠버네티스 기능과 무엇이 비슷한가?

1. VMI(VirtualMachineInstance) 를 대상으로 함
2. 지정한 replicas 수를 항상 유지
3. 하나가 죽으면 새 VMI를 자동 생성
4. 기본적으로 쿠버네티스의 ReplicaSet과 동일



가상머신 복제(vmirs) 설명 2

직접적으로 가상머신을 생성하지 않고, `replicas:`의 개수를 유지하려고 한다. 그리고, 이 기능의 치명적(?)인 단점은 다음과 같다.

1. 디스크 문제

- VMIRS는 공유 디스크 전제
- 대부분 RWX PVC 필요
- VM별 디스크 분리 불가능

VMIRS (디스크 공유)

```
├─ VMI #1 (Running)
├─ VMI #2 (Running)
└─ VMI #3 (Running)
```



가상머신 복제(vmirs) 설명 3

2. 아래와 같은 관리 기능은 사용이 불가능

- Snapshot
- Clone
- Start/Stop

3 .Live Migration과 궁합이 나쁨

- Pod 네트워크 제약이 많음(마이그레이션은 불가능)
- 장애 시 마이그레이션이 아니라 재생성



가상머신 복제(vmirs) 예제 1

```
cat <<EOF> | kubectl apply -
apiVersion: kubevirt.io/v1
kind: VirtualMachineInstanceReplicaSet
metadata:
  name: cirros-vmirs
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: cirros
```



가상머신 복제(vmirs) 예제 2

```
template:  
  metadata:  
    labels:  
      app: cirros  
  spec:  
    instancetype:  
      name: medium
```



가상머신 복제(vmirs) 예제 3

```
domain:
  devices:
    disks:
      - name: rootdisk
        disk:
          bus: virtio
    interfaces:
      - name: default
        masquerade: {}
```



가상머신 복제(vmirs) 예제 4

```
networks:  
  - name: default  
    pod: {}  
volumes:  
  - name: rootdisk  
    persistentVolumeClaim:  
      claimName: cirros-rwx-pvc
```

EOF



가상머신 복제(vmirs) 예상 결과

```
kubectl get vmirs
```

NAME	DESIRED	CURRENT	READY	AGE
cirros-vmirs	3	3	3	30s

```
kubectl get vmi -l app=cirros -o wide
```

NAME	PHASE	IP	NODE
cirros-vmirs-xxxxx	Running	10.244.0.51	node-1
cirros-vmirs-yyyyy	Running	10.244.1.12	node-2
cirros-vmirs-zzzzz	Running	10.244.2.33	node-3



가상머신 내보내기(vmexport) 설명

VirtualMachineExport (vmexport), KubeVirt에서 VM(또는 PVC)의 디스크를 다운로드 가능한 형태로 내보내는(Export) 기능.

이 기능은 VM 백업/다른 환경으로 이동 시 사용한다. 백업 시, 두 가지 방법으로 백업이 가능하다.

1. VM (VM에 연결된 볼륨을 export)
2. PVC (특정 디스크/PVC를 직접 export)

이미지 파일로 다운로드 받기 위해서는 생성 후 제공해주는 URL로 다운로드가 가능하다. 가상머신 운영 부분에서 자세히 다루도록 한다.



가상머신 사양 설명(vmf)

VirtualMachineInstanceType은 VM의 하드웨어 스펙(Flavor)을 정의한다. 오픈스택에서 Flavor 기능과 동일하다. 선언이 안되어 있는 경우 가상머신 생성 시 명시해야 한다.

1. 인스턴스 사양 선언
2. 직접적으로 VM/VMI에 사양 선언

실 서비스 구성 시, 가급적이면 vmf 기반으로 구성한다.



가상머신 사양 직접 명시 예제

아래는 앞에서 구성 시 사용하였던 직접 하드웨어 사양 명시.

```
spec:
  running: true
  template:
    spec:
      domain:
        resources:
          requests:
            cpu: "1"
            memory: 1Gi
```



인스턴스 타입 선언(vmf)

아래와 같이 인스턴스 타입을 선언 및 생성한다.

```
cat <<'EOF' | kubectl apply -f -
apiVersion:instancetype.kubevirt.io/v1beta1
kind: VirtualMachineInstancetype
metadata:
  name: medium
  namespace: default
spec:
  cpu:
    guest: 2
  memory:
    guest: 4Gi
EOF
```



가상머신 사양 선언(vmf)

선언이 되어 있는 경우, 아래와 같이 하드웨어 사양 선언.

```
kubectl get vmf
```

```
NAME      AGE
```

```
medium    52s
```

```
spec:
```

```
  instancetype:
```

```
    name: small
```



가상머신 권장 설정 설명(VMP)

VMP는 VMF와 비슷하지만, 차이점은 사용하는 가상머신 이미지에 따라서 자동으로 가상머신 사양을 선택한다. 오픈스택과 비교하면, Flavor에 메타정보를 오버라이드 하는 기능과 동일하다.

예를 들어 다음과 같이 VMF를 설정한다.

- 디스크 버스 (virtio/sata)
- NIC 모델
- 머신 타입 (q35 등)
- OS 타입 힌트 (linux/windows)

이 기능은 권장사항, 강제적으로 적용되는 자원은 아니다. 사용자에게 좀 더 나은 하드웨어 선택 환경을 제공한다.

- `preferredDiskBus: virtio`
- `preferredInterfaceModel: virtio`



가상머신 권장 설정 선언(VMP)

아래와 같이 가상머신 사양 덮어쓰기가 가능하다.

```
cat <<'EOF' | kubectl apply -f -  
apiVersion: instancetype.kubevirt.io/v1beta1  
kind: VirtualMachinePreference  
metadata:  
  name: linux  
  namespace: default
```



가상머신 권장 설정 선언(VMP)

아래와 같이 가상머신 사양 덮어쓰기가 가능하다.

```
spec:
  devices:
    preferredDiskBus: virtio
    preferredInterfaceModel: virtio
  machine:
    preferredMachineType: q35
  firmware:
    preferredUseEfi: false
```

EOF



가상머신 권장 설정 선언 확인(VMP)

아래와 같이 확인이 가능하다. 사용 방법은 가상머신 운영 부분에서 다시 다루도록 한다.

```
kubectl get vmp
```

```
NAME    AGE
```

```
linux   46s
```



VMF/VMP 비교

구분	InstanceType (vmf)	Preference (vmp)
역할	스펙 정의	권장 설정
성격	강제	힌트
CPU / Memory	포함	없음
디바이스 설정	불가능	가능
오버라이드	불가능	가능
선택 가능	강제	선택사항
OpenStack 대응	Flavor	Image metadata



오픈스택과 자원 비교

KubeVirt/CDI	의미	OpenStack 대응
DataVolume(dv)	이미지 → PVC로 가져오기	Glance 이미지로 Cinder 볼륨 생성(유사)
PVC	VM 디스크	Cinder Volume
VM(virtualmachine)	VM 정의 + 전원	Nova Server Status
VMI(virtualmachineinstance)	실행 인스턴스 런타임	Nova instance
vmsnapshot/vmrestore	스냅샷/복원	volume snapshot/rollback 또는 server image
vmclone	VM 복제	이미지 기반 새 서버 생성(유사)
vmpool/vmirs	VM 스케일아웃	Heat AutoScaling(유사)
StorageClass/StorageProfile	스토리지 정책	Cinder backend/type



마지막 virtctl 명령어

virtctl 명령어는 가상머신 구성 시 자원 설정 및 수정을 위해서 사용하는 명령어. 이 두 가지 명령어는 다음과 같은 차이가 있다. 이 명령어에 대해서는 교육을 진행하면서 사용한다.

1. kubect, 리소스 관리 및 조작
2. virtctl, VM 조작(운영에 더 가까움)

구분	kubectl	virtctl
역할	리소스 관리	VM 조작
대상	CR, YAML	실행 중 VM
콘솔	미지원	가능
전원 제어	미지원	가능
마이그레이션	미지원	가능
교육 난이도	높음	낮음

